

laboratorio_embeddings.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todo

RAM Disco

Transforma textos en embeddings

Estudiante: Harold Daniel Duque - CC 1033726983

Curso: Procesamiento de Lenguaje Natural

Este laboratorio muestra el recorrido desde la tokenización hasta la comparación semántica de oraciones. La primera parte usa TextVectorization y una capa Embedding de TensorFlow. La segunda usa un modelo multilingüe preentrenado para comparar frases completas.

Mapa rápido: qué se está haciendo y para qué

La meta no es que el computador lea como una persona. La idea es pasar cada ticket a números que conserven parte de su significado y usarlos para ordenar solicitudes de servicio al cliente.

Ticket escrito por un cliente o solicitud

↓

Embedding: vector numérico del texto
(permite comparar el significado aproximado entre solicitudes)

↓

30 tickets con tema de referencia
(ejemplos cuyo tema esperado ya conocemos: entregas, pagos o cuenta)

↓

Clasificador aprende qué tipo de vector suele corresponder a cada tema
(así puede proponer una ruta para solicitudes nuevas)

↓

Ticket nuevo → tema propuesto + confianza
(la confianza es la probabilidad calculada, no una certeza ni un peso)

↓

Confianza alta: ruta sugerida | Duda: revisión humana (rojo)
(rojo llama la atención: no se enruta automáticamente hasta que alguien lo revise)

Un embedding no es un número único ni una palabra codificada; es una lista de números, como coordenadas, que permite ubicar textos parecidos cerca entre sí. Los 30 temas de referencia son la respuesta esperada de los ejemplos semilla y representan la etiqueta que pondría una persona en esta simulación. El clasificador observa esos ejemplos para aprender una relación entre vector y tema.

La confianza no es un peso adicional ni demuestra que el resultado sea correcto. Es la probabilidad más alta entre las opciones que calcula el clasificador. Si es alta y la segunda opción queda lejos, se puede sugerir una ruta. Si es baja, o si dos opciones quedan casi empatadas, el caso se pinta de rojo para que nadie lo dé por clasificado sin leerlo. La revisión humana corrige la ruta y, más adelante, esa corrección puede convertirse en un nuevo ejemplo de referencia.

El embedding se calcula antes de clasificar. El resultado del clasificador no es otro embedding: es una etiqueta, por ejemplo pagos , y un nivel de confianza. Después, los mismos embeddings también se reutilizan para agrupar tickets parecidos y dibujarlos en un espacio 3D.

1. Instalación

En Google Colab, ejecute esta celda una sola vez. La descarga del modelo puede tardar unos minutos la primera vez.

[1] 8s

!pip -q install sentence-transformers seaborn plotly

2. Importaciones y corpus

El corpus simula tickets de una tienda virtual. La muestra tiene 30 tickets iniciales etiquetados, 15 tickets de validación y 15 tickets operativos sin etiqueta. Los datos son sintéticos: representan cómo podría organizarse una revisión humana, no registros reales de clientes.

[2] 1m

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import tensorflow as tf
from tensorflow.keras import layers
from IPython.display import display
from sentence_transformers import SentenceTransformer
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

# Una semilla fija hace más fácil repetir los resultados del ejemplo.
SEED = 7
np.random.seed(SEED)
tf.random.set_seed(SEED)

# 30 tickets revisados y etiquetados manualmente en esta simulación.
corpus = np.array([
    'El seguimiento del pedido no cambia desde hace tres días.',
    'Mi compra aparece enviada, pero el rastreo no muestra movimientos.',
    'El mensajero marcó la entrega como realizada y nadie recibió el paquete.',
    'Necesito cambiar la dirección porque el pedido todavía no ha salido.',
    'La fecha prometida pasó y mi compra aún no llega.',
    'Quiero saber dónde está el producto que compré ayer.',
    'El transportador cambió la fecha de entrega sin avisarme.',
    'Me llegó una caja, pero falta uno de los productos del pedido.',
    'El paquete llegó roto y necesito reportar la novedad.',
    'Prefiero recoger mi compra en un punto cercano.',
    'El cobro de mi pedido apareció dos veces en la tarjeta.',
    'Cancelé la compra, pero todavía no veo el reembolso.',
    'La tarjeta fue rechazada aunque tiene saldo disponible.',
    'Pagué por transferencia y el pedido sigue pendiente de pago.',
    'La factura tiene un valor diferente al que mostraba el carrito.',
    'El cupón de descuento no se aplicó al momento de pagar.',
    'El pago por PSE fue aprobado, pero el pedido no aparece confirmado.',
    'La compra se canceló después de usar la billetera digital.',
    'Quiero saber por qué una cuota quedó más alta de lo esperado.',
    'Necesito descargar la factura de una compra anterior.',
    'No me llega el código OTP para entrar a mi cuenta.',
    'Mi usuario quedó bloqueado después de varios intentos.',
    'Olvidé la contraseña y el enlace para recuperarla no funciona.',
    'Quiero actualizar el número de teléfono asociado a mi perfil.',
    'La aplicación se cierra apenas intento iniciar sesión.',
    'No reconozco el dispositivo que aparece conectado a mi cuenta.',
    'Necesito cambiar el correo electrónico de mi cuenta.',
    'El código de verificación de dos pasos siempre sale inválido.',
    'Quedaron dos perfiles creados con el mismo documento.',
    'El nombre que aparece en mi perfil está escrito de forma incorrecta.'
])

etiquetas = np.array([0] * 10 + [1] * 10 + [2] * 10)
```

```
nombres_clase = ['entregas', 'pagos', 'cuenta']

# Estos 15 tickets tienen una etiqueta conocida solo para evaluar el modelo.
tickets_validacion = np.array([
    'El rastreo sigue detenido y no sé cuándo llegará la compra.',
    'La plataforma indicó que mi pedido fue entregado, pero no lo recibí.',
    'El envío quedó devuelto al vendedor sin que yo lo solicitara.',
    'El paquete lleva una semana en la misma ciudad.',
    'Necesito confirmar si todavía puedo modificar la dirección del pedido.',
    'Veo un débito duplicado por la misma compra.',
    'El reembolso prometido no se refleja en mi cuenta bancaria.',
    'El total final cambió cuando escogí pagar con tarjeta.',
    'La entidad autorizó el pago, pero la tienda lo rechazó.',
    'El pedido figura sin pagar pese a que recibí el comprobante.',
    'No recibo el mensaje con el código para ingresar.',
    'Mi cuenta se bloqueó y no puedo hacer compras.',
    'El enlace para cambiar mi clave ya venció.',
    'No puedo iniciar sesión desde el celular nuevo.',
    'Aparece un inicio de sesión que yo no realicé.'
])
etiquetas_validacion = np.array([0] * 5 + [1] * 5 + [2] * 5)

# Lote operativo: el sistema recibe estas solicitudes sin etiqueta.
tickets_operativos = np.array([
    'Mi pedido sale como despachado desde ayer, pero no hay ninguna actualización.',
    'El repartidor dejó el paquete en otra dirección.',
    '¿Puedo cambiar el lugar de entrega antes de que salga la compra?',
    'Me faltó un artículo dentro de la caja que recibí.',
    'El envío tiene retraso y nadie me confirma una fecha nueva.',
    'Me cobraron dos veces el mismo producto.',
    'Devolví el artículo y todavía no aparece el dinero.',
    'El valor cobrado no coincide con el resumen del pedido.',
    'Mi transferencia fue exitosa, pero el pago sigue en revisión.',
    'El pedido quedó pendiente después de que realicé el pago.',
    'La clave nueva no me permite entrar a la aplicación.',
    'El código para validar la cuenta nunca llega a mi celular.',
    'Necesito sacar un dispositivo desconocido de mi perfil.',
    'No puedo cambiar el correo asociado a mi usuario.',
    'El pedido cambió de dirección y ahora el pago no aparece.'
])

# Esta tabla permite inspeccionar el conjunto de referencia antes de entrenar.
pd.DataFrame({'texto': corpus, 'clase_revisada': [nombres_clase[i] for i in etiquetas]})
```

	texto	clase_revisada
0	El seguimiento del pedido no cambia desde hace...	entregas
1	Mi compra aparece enviada, pero el rastreo no ...	entregas
2	El mensajero marcó la entrega como realizada y...	entregas
3	Necesito cambiar la dirección porque el pedido...	entregas
4	La fecha prometida pasó y mi compra aún no llega.	entregas
5	Quiero saber dónde está el producto que compré...	entregas
6	El transportador cambió la fecha de entrega si...	entregas
7	Me llegó una caja, pero falta uno de los produ...	entregas
8	El paquete llegó roto y necesito reportar la n...	entregas
9	Prefiero recoger mi compra en un punto cercano.	entregas
10	El cobro de mi pedido apareció dos veces en la...	pagos
11	Cancelé la compra, pero todavía no veo el reem...	pagos
12	La tarjeta fue rechazada aunque tiene saldo di...	pagos
13	Pagué por transferencia y el pedido sigue pend...	pagos
14	La factura tiene un valor diferente al que mos...	pagos
15	El cupón de descuento no se aplicó al momento ...	pagos
16	El pago por PSE fue aprobado, pero el pedido n...	pagos
17	La compra se canceló después de usar la billet...	pagos
18	Quiero saber por qué una cuota quedó más alta ...	pagos
19	Necesito descargar la factura de una compra an...	pagos
20	No me llega el código OTP para entrar a mi cue...	cuenta
21	Mi usuario quedó bloqueado después de varios l...	cuenta
22	Olvíde la contraseña y el enlace para recupera...	cuenta
23	Quiero actualizar el número de teléfono asocia...	cuenta
24	La aplicación se cierra apenas intento iniciar...	cuenta
25	No reconozco el dispositivo que aparece conect...	cuenta
26	Necesito cambiar el correo electrónico de mi c...	cuenta
27	El código de verificación de dos pasos siempre...	cuenta
28	Quedaron dos perfiles creados con el mismo doc...	cuenta
29	El nombre que aparece en mi perfil está escrit...	cuenta

3. Tokenización con TextVectorization

Esta función construye un vocabulario local. Es importante separar este paso del embedding: aquí cada palabra recibe un índice, pero todavía no se está midiendo su significado.

```
[3] ✓ 1s
def crear_vectorizador(textos, max_tokens=500, longitud=14):
    vectorizador = layers.TextVectorization(
        max_tokens=max_tokens,
        standardize='lower_and_strip_punctuation',
        split='whitespace',
        output_mode='int',
        output_sequence_length=longitud,
    )
    # 'adapt' aprende solamente el vocabulario de los tickets semilla.
    vectorizador.adapt(textos)
    return vectorizador

# Este vectorizador no entiende todavía el significado; solo asigna índices.
vectorizador = crear_vectorizador(corpus)
vocabulario = vectorizador.get_vocabulary()
print(f'Tamaño del vocabulario: {len(vocabulario)}')
print(f'Primeros 20 tokens:', vocabulario[:20])

ejemplo = tf.constant(['El pedido no cambia en el seguimiento'])
secuencia = vectorizador(ejemplo)
print(f'Secuencia de índices:', secuencia.numpy()[0])

# La palabra inventada queda asociada al token de desconocido del vocabulario local.
prueba_desconocida = tf.constant(['El token passkey no funciona en mi perfil'])
print(f'Prueba con término poco frecuente:', vectorizador(prueba_desconocida).numpy()[0])
```

Tamaño del vocabulario: 167
Primeros 20 tokens: ['', '[UNK]', np.str_('el'), np.str_('de'), np.str_('la'), np.str_('no'), np.str_('mi'), np.str_('pedido'), np.str_('compra'), np.str_('y'), np.str_('que'), np.str_('pero'), np.str_('nec
Secuencia de índices: [2 7 5 150 18 2 61 0 0 0 0 0 0]

Prueba con término poco frecuente: [2 1 1 5 113 18 6 28 0 0 0 0 0 0]

4. Embedding aprendido dentro de Keras

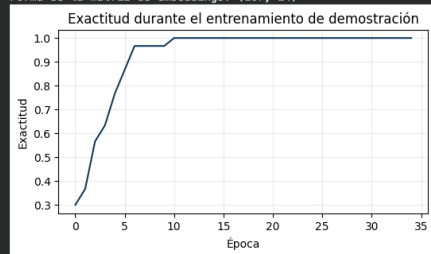
El siguiente modelo clasifica los textos por tema. La capa `Embedding` convierte los índices en vectores densos de 24 dimensiones. El resultado del entrenamiento sirve para inspeccionar el flujo, no para afirmar que el modelo generaliza: el corpus es pequeño y no se usa una partición de prueba independiente.

```
[4]: def crear_modelo_embedding(vectorizador, dimension=24, clases=3):
    entrada = tf.keras.Input(shape=(1,), dtype=tf.string, name='texto')
    # El texto se vuelve una secuencia de índices antes de entrar a Embedding.
    indices = vectorizador(entrada)
    vectores = layers.Embedding(
        input_dim=len(vectorizador.get_vocabulary()),
        output_dim=dimension,
        mask_zero=True,
        name='embedding_palabras',
    )(indices)
    # Se resume una secuencia de vectores en un solo vector por ticket.
    resumen = layers.GlobalAveragePooling1D()(vectores)
    salida = layers.Dense(clases, activation='softmax')(resumen)
    modelo = tf.keras.Model(entrada, salida)
    modelo.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return modelo

# Esta parte es una demostración de un embedding aprendido desde cero.
# El flujo operativo posterior usará embeddings preentrenados de oraciones.
modelo_local = crear_modelo_embedding(vectorizador)
# Keras 3 no recibe directamente arreglos NumPy con dtype Unicode.
textos_modelo = tf.constant([[texto] for texto in corpus.tolist()], dtype=tf.string)
etiquetas_modelo = tf.constant(etiquetas, dtype=tf.int32)
historial = modelo_local.fit(textos_modelo, etiquetas_modelo, epochs=35, verbose=0)
perdida, exactitud = modelo_local.evaluate(textos_modelo, etiquetas_modelo, verbose=0)
print(f'Exactitud sobre el corpus de demostración: {exactitud:.3f}')
print(f'Forma de la matriz de embeddings:', modelo_local.get_layer('embedding_palabras').get_weights()[0].shape)

plt.figure(figsize=(6, 3))
plt.plot(historial.history['accuracy'], color='#153A5B')
plt.title('Exactitud durante el entrenamiento de demostración')
plt.xlabel('Época')
plt.ylabel('Exactitud')
plt.grid(alpha=0.25)
plt.show()
```

Exactitud sobre el corpus de demostración: 1.000
Forma de la matriz de embeddings: (167, 24)



5. Embeddings de oraciones y similitud coseno

El modelo multilingüe trabaja con oraciones completas. La prueba incluye dos tickets sobre entrega, dos sobre pagos y dos sobre acceso a cuenta. El mapa de calor, el espacio 3D y el CSV resultante permiten revisar los resultados de varias maneras.

```
[5]: oraciones_prueba = [
    'El pedido no se mueve desde hace tres días en el seguimiento.',
    'El rastreo de mi compra sigue igual y no registra avances.',
    'El cobro apareció dos veces en mi tarjeta.',
    'Cancelé la compra y aún no recibo el reembolso.',
    'No llega el código para entrar en mi cuenta.',
    'Mi usuario quedó bloqueado después de varios intentos.'
]

def comparar_oraciones(oraciones, nombre_modelo='paraphrase-multilingual-MiniLM-L12-v2'):
    # El modelo ya fue entrenado previamente para representar oraciones.
    codificador = SentenceTransformer(nombre_modelo)
    embeddings = codificador.encode(oraciones, normalize_embeddings=True, show_progress_bar=True)
    # Con vectores normalizados, el producto punto equivale a similitud coseno.
    matriz = embeddings @ embeddings.T
    etiquetas_cortas = [f'0{i+1}' for i in range(len(oraciones))]
    tabla = pd.DataFrame(matriz, index=etiquetas_cortas, columns=etiquetas_cortas)
    return embeddings, tabla

def graficar_similitud(tabla):
    plt.figure(figsize=(8, 6))
    sns.heatmap(tabla, annot=True, fmt='.3f', cmap='YlGnBu', vmin=-1, vmax=1, square=True)
    plt.title('Similitud coseno entre oraciones')
    plt.show()

embeddings_oracion, tabla_similitud = comparar_oraciones(oraciones_prueba)
graficar_similitud(tabla_similitud)
tabla_similitud
```

models.sentencetransformers: downloading bytes: 440MB, 30.5MB/s

Loading weights: 100% 199/199 [00:00<00:00, 855.50it/s]

tokenizer_config.json: 100% 526/526 [00:00<00:00, 10.0kB/s]

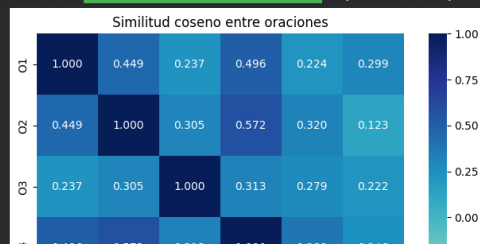
tokenizer.json: reconstructing file: 100% 9.08MB / 9.08MB, 893kB/s

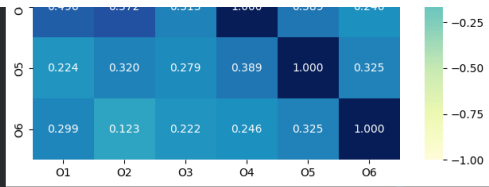
tokenizer.json: downloading bytes: 5.42MB, 530kB/s

special_tokens_map.json: 100% 239/239 [00:00<00:00, 2.86kB/s]

config.json: 100% 190/190 [00:00<00:00, 8.40kB/s]

Batches: 100% 1/1 [00:01<00:00, 1.10s/it]





	01	02	03	04	05	06
01	1.000000	0.448625	0.237172	0.495707	0.224059	0.299059
02	0.448625	1.000000	0.305437	0.572233	0.320046	0.123002
03	0.237172	0.305437	1.000000	0.312954	0.278917	0.221636
04	0.495707	0.572233	0.312954	1.000000	0.389485	0.245648
05	0.224059	0.320046	0.278917	0.389485	1.000000	0.325297
06	0.299059	0.123002	0.221636	0.245648	0.325297	1.000000

Próximos pasos: [Generar código con tabla_similitud](#) [New interactive sheet](#)

6. Mini ejemplo en 3D: solo seis tickets

Esta gráfica usa únicamente las seis oraciones O1-O6 de la sección de similitud; sirve para entender la idea sin llenar la pantalla de puntos. No representa el conjunto completo. El mapa con los 60 tickets, las revisiones humanas y los errores de validación aparece al final, después de la sección 7. Cada oración original tiene muchas dimensiones. Para poder verla, aplicamos PCA y la proyectamos en tres ejes. Los puntos cercanos sugieren que el modelo ubicó esas frases en una zona parecida. Es una proyección visual, no una prueba absoluta de que dos tickets sean equivalentes.

```
(6)
✓ 9 s

temas_prueba = ['Entrega', 'Entrega', 'Pago', 'Pago', 'Cuenta', 'Cuenta']

def visualizar_espacio_3d(embeddings, oraciones, temas):
    # PCA conserva la mayor variación posible al pasar del vector original a tres ejes.
    proyeccion = PCA(n_components=3, random_state=SEED).fit_transform(embeddings)
    puntos = pd.DataFrame({
        'x': proyeccion[:, 0],
        'y': proyeccion[:, 1],
        'z': proyeccion[:, 2],
        'tema': temas,
        'ticket': [f'O{i + 1}' for i in range(len(oraciones))],
        'texto': oraciones,
    })
    figura = px.scatter_3d(
        puntos, x='x', y='y', z='z', color='tema', text='ticket',
        hover_data={'texto': True, 'x': ':.3f', 'y': ':.3f', 'z': ':.3f'},
        color_discrete_map={'Entrega': '#2E8B57', 'Pago': '#D99B22', 'Cuenta': '#7655A5'},
        title='Mini ejemplo 3D: seis tickets según sus embeddings',
    )
    figura.update_traces(marker={'size': 7}, textposition='top center')
    figura.update_layout(scene={'xaxis_title': 'Componente 1', 'yaxis_title': 'Componente 2', 'zaxis_title': 'Componente 3'})
    figura.show()
    return puntos

puntos_3d = visualizar_espacio_3d(embeddings_oracion, oraciones_prueba, temas_prueba)
puntos_3d
```

Mini ejemplo 3D: seis tickets según sus embeddings



	x	y	z	tema	ticket	texto
0	0.288820	0.429618	-0.286884	Entrega	O1	El pedido no se mueve desde hace tres días en ...
1	0.503028	-0.065030	0.104037	Entrega	O2	El rastreo de mi compra sigue igual y no regis...
2	-0.155038	-0.653370	-0.437708	Pago	O3	El cobro apareció dos veces en mi tarjeta.
3	0.344020	0.071376	0.135259	Pago	O4	Cancelé la compra y aún no recibo el reembolso.
4	-0.295376	-0.172816	0.630502	Cuenta	O5	No llega el código para entrar en mi cuenta.
5	-0.665455	0.390222	-0.145207	Cuenta	O6	Mi usuario quedó bloqueado después de varios L...

Próximos pasos: [Generar código con puntos_3d](#) [New interactive sheet](#)

Cómo interpretar la matriz

- O1 y O2 deben quedar cerca porque describen el mismo problema de seguimiento con palabras distintas.
- O3 y O4 pertenecen a pagos, pero no describen exactamente el mismo caso: uno habla de un cobro duplicado y el otro de un reembolso pendiente.
- O5 y O6 deberían formar otra cercanía porque los dos son problemas de acceso a cuenta.
- En la vista 3D es normal que haya pequeñas diferencias frente al mapa de calor: PCA reduce muchas dimensiones a tres ejes para poder mostrar los puntos.

```
(7)
✓ 0 s

def pares_mas_similares(tabla, oraciones, cantidad=5):
    resultados = []
    # Solo se revisa una vez cada pareja para no duplicar A-B y B-A.
    for i in range(len(oraciones)):
        for j in range(i + 1, len(oraciones)):
            resultados.append({
                'oración_a': oraciones[i],
                'oración_b': oraciones[j],
                'similitud_coseno': float(tabla.iloc[i, j]),
            })
    return pd.DataFrame(resultados).sort_values('similitud_coseno', ascending=False).head(cantidad)
```



```
resumen_pares = pares_mas_similares(tabla_similitud, oraciones_prueba)
display(resumen_pares)

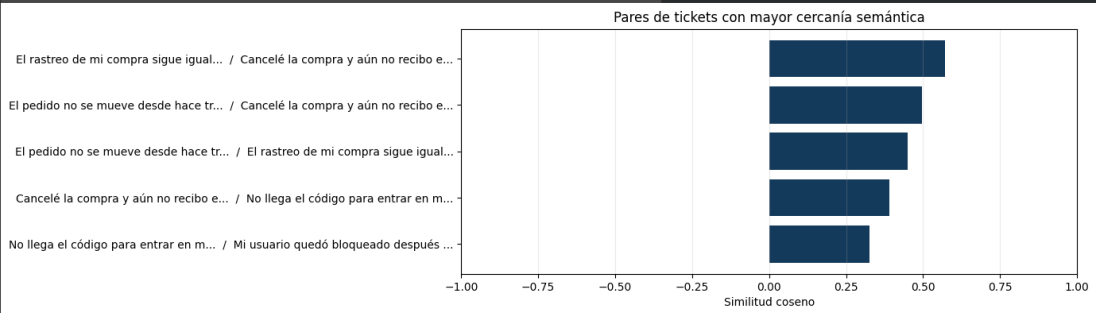
def graficar_pares(pares):
    pares_grafica = pares.sort_values('similitud_coseno')
    etiquetas = [{"fila.oración_a[:35]}... / {fila.oración_b[:35]}..." for _, fila in pares_grafica.iterrows()]
    plt.figure(figsize=(10, 4))
    plt.barh(etiquetas, pares_grafica['similitud_coseno'], color='#153A5B')
    plt.xlim(-1, 1)
    plt.xlabel('Similitud coseno')
    plt.title('Pares de tickets con mayor cercanía semántica')
    plt.grid(axis='x', alpha=0.25)
    plt.show()

graficar_pares(resumen_pares)

def guardar_resultados(tabla, ruta='resultados_similitud.csv'):
    tabla.to_csv(ruta, encoding='utf-8')
    print(f'Resultados guardados en: {ruta}')

guardar_resultados(tabla_similitud)
```

	oración_a	oración_b	similitud_coseno
6	El rastreo de mi compra sigue igual y no regis...	Cancelé la compra y aún no recibo el reembolso.	0.572233
2	El pedido no se mueve desde hace tres días en ...	Cancelé la compra y aún no recibo el reembolso.	0.495707
0	El pedido no se mueve desde hace tres días en ...	El rastreo de mi compra sigue igual y no regis...	0.448625
12	Cancelé la compra y aún no recibo el reembolso.	No llega el código para entrar en mi cuenta.	0.389485
14	No llega el código para entrar en mi cuenta.	Mi usuario quedó bloqueado después de varios L...	0.325297



Resultados guardados en: resultados_similitud.csv

Próximos pasos: [Generar código con resumen_pares](#) [New interactive sheet](#)

7. Flujo de clasificación con revisión humana simulada

Esta parte lleva el ejemplo a un escenario más parecido al de una empresa. Los 30 tickets semilla están **etiquetados de referencia** y representan el trabajo que haría una persona del equipo de servicio al cliente. No son datos reales ni una revisión que se haya realizado fuera del laboratorio. Con esos ejemplos, se entrenará un clasificador sobre embeddings.

Los 15 tickets de validación sí tienen etiqueta, pero el clasificador no los ve durante el ajuste; sirven para medir qué tan razonable sale la propuesta. Los 15 tickets operativos llegan sin tema. Allí el sistema puede sugerir una categoría, pero una baja confianza o una decisión muy pareja se marca en rojo y pasa a verificación humana.

```
# El embedding se calcula antes de entrenar el clasificador de temas.
modelo_semantico = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
embeddings_entrenamiento = modelo_semantico.encode(
    corpus.tolist(), normalize_embeddings=True, show_progress_bar=True
)

embeddings_validacion = modelo_semantico.encode(
    tickets_validacion.tolist(), normalize_embeddings=True, show_progress_bar=True
)

clasificador_semantico = LogisticRegression(
    max_iter=2000, random_state=SEED, class_weight='balanced'
)
clasificador_semantico.fit(embeddings_entrenamiento, etiquetas)
prediccion_validacion = clasificador_semantico.predict(embeddings_validacion)

print('Resultados sobre los 15 tickets separados para validación:')
print(classification_report(
    etiquetas_validacion, prediccion_validacion,
    target_names=nombres_clase, zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    etiquetas_validacion, prediccion_validacion,
    display_labels=nombres_clase, cmap='Blues', colorbar=False
)
plt.title('Validación del clasificador basado en embeddings')
plt.show()
```

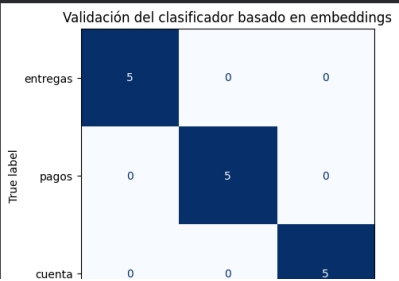
Loading weights: 100% 199/199 [00:00<00:00, 1720.12B/s]

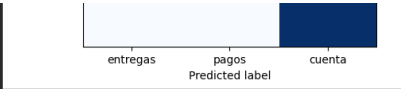
Batches: 100% 1/1 [00:00<00:00, 11.64B/s]

Batches: 100% 1/1 [00:00<00:00, 12.33B/s]

Resultados sobre los 15 tickets separados para validación:

	precision	recall	f1-score	support
entregas	1.00	1.00	1.00	5
pagos	1.00	1.00	1.00	5
cuenta	1.00	1.00	1.00	5
accuracy			1.00	15
macro avg	1.00	1.00	1.00	15
weighted avg	1.00	1.00	1.00	15





Clasificación del lote operativo

La confianza es la probabilidad más alta que entrega el clasificador; no es una garantía de que la respuesta sea correcta. Además se mira el margen entre la primera y la segunda categoría. Si las dos opciones quedan muy cerca, el caso también se considera dudoso. Los valores de 0,70 y 0,18 son reglas de demostración: una empresa debería definirlos con datos históricos, costos de error y revisión de calidad.

```
[9]
✓ Os
embeddings_operativos = modelo_semantico.encode(
    tickets_operativos.tolist(), normalize_embeddings=True, show_progress_bar=True
)
probabilidades = clasificador_semantico.predict_proba(embeddings_operativos)
predicciones_operativas = clasificador_semantico.predict(embeddings_operativos)
confianza = probabilidades.max(axis=1)
probabilidades_ordenadas = np.sort(probabilidades, axis=1)
margen = probabilidades_ordenadas[:, -1] - probabilidades_ordenadas[:, -2]

UMBRAL_CONFIANZA = 0.70
MARGEN_MINIMO = 0.18
requiere_revision = (confianza < UMBRAL_CONFIANZA) | (margen < MARGEN_MINIMO)

resultados_operativos = pd.DataFrame({
    'ticket': [f'OP{i + 1:02d}' for i in range(len(tickets_operativos))],
    'texto': tickets_operativos,
    'tema_propuesto': [nombres_clase[indice] for indice in predicciones_operativas],
    'confianza': confianza.round(3),
    'margen_entre_opciones': margen.round(3),
    'estado': np.where(requiere_revision, 'Revisión humana', 'Clasificación propuesta'),
})

def resaltar_revision(fila):
    if fila['estado'] == 'Revisión humana':
        return ['background-color: #FDECEC; color: #A61B1B; font-weight: bold' for _ in fila]
    return ['' for _ in fila]

print(f'Casos enviados a revisión: {requiere_revision.sum()} de {len(resultados_operativos)}')
display(resultados_operativos.style.apply(resaltar_revision, axis=1))

# El CSV deja la evidencia de qué se propuso y qué casos deben ser verificados.
resultados_operativos.to_csv('resultados_clasificacion_operativa.csv', index=False, encoding='utf-8')
```

Batches: 100% 1/1 [00:00<00:00, 16.07W/s]

Casos enviados a revisión: 15 de 15

	ticket	texto	tema_propuesto	confianza	margen_entre_opciones	estado
0	OP01	Mi pedido sale como despachado desde ayer, pero no hay ninguna actualización.	entregas	0.545000	0.296000	Revisión humana
1	OP02	El repartidor dejó el paquete en otra dirección.	entregas	0.402000	0.010000	Revisión humana
2	OP03	¿Puedo cambiar el lugar de entrega antes de que salga la compra?	entregas	0.595000	0.323000	Revisión humana
3	OP04	Me faltó un artículo dentro de la caja que recibí.	entregas	0.442000	0.141000	Revisión humana
4	OP05	El envío tiene retraso y nadie me confirma una fecha nueva.	entregas	0.616000	0.385000	Revisión humana
5	OP06	Me cobraron dos veces el mismo producto.	pagos	0.500000	0.221000	Revisión humana
6	OP07	Devolví el artículo y todavía no aparece el dinero.	pagos	0.440000	0.081000	Revisión humana
7	OP08	El valor cobrado no coincide con el resumen del pedido.	pagos	0.591000	0.373000	Revisión humana
8	OP09	Mi transferencia fue exitosa, pero el pago sigue en revisión.	pagos	0.551000	0.296000	Revisión humana
9	OP10	El pedido quedó pendiente después de que realicé el pago.	pagos	0.550000	0.262000	Revisión humana
10	OP11	La clave nueva no me permite entrar a la aplicación.	cuenta	0.627000	0.417000	Revisión humana
11	OP12	El código para validar la cuenta nunca llega a mi celular.	cuenta	0.586000	0.361000	Revisión humana
12	OP13	Necesito sacar un dispositivo desconocido de mi perfil.	cuenta	0.655000	0.434000	Revisión humana
13	OP14	No puedo cambiar el correo asociado a mi usuario.	cuenta	0.663000	0.479000	Revisión humana
14	OP15	El pedido cambió de dirección y ahora el pago no aparece.	entregas	0.388000	0.024000	Revisión humana

Agrupar antes de poner una etiqueta

Aquí KMeans no recibe los nombres de los temas: solo ve los embeddings y reúne textos cercanos en tres grupos. Después se comparan esos grupos con los tickets de referencia para entender qué parece representar cada uno. Esa comparación no convierte al grupo en una verdad automática; una persona todavía debe revisar ejemplos y decidir si el tema, el número de grupos y la ruta de atención son adecuados.

```
[18]
✓ Os
textos_todos = np.concatenate([corpus, tickets_validacion, tickets_operativos])
embeddings_todos = np.vstack([
    embeddings_entrenamiento, embeddings_validacion, embeddings_operativos
])
origen_todos = ([f'semilla revisada' * len(corpus) +
    f'validación' * len(tickets_validacion) +
    f'operativo sin etiqueta' * len(tickets_operativos)])

agrupador = KMeans(n_clusters=3, n_init=20, random_state=SEED)
clusters = agrupador.fit_predict(embeddings_todos)
resultados_agrupamiento = pd.DataFrame({
    'texto': textos_todos,
    'origen': origen_todos,
    'cluster_propuesto': [f'Grupo {cluster + 1}' for cluster in clusters],
})

# Solo se usan las etiquetas conocidas para interpretar los grupos después de crearlos.
temas_referencia = [nombres_clase[indice] for indice in np.concatenate([etiquetas, etiquetas_validacion])]
referencia_clusters = resultados_agrupamiento.iloc[:len(temas_referencia)].copy()
referencia_clusters['tema_de_referencia'] = temas_referencia
tabla_clusters = pd.crosstab(
    referencia_clusters['cluster_propuesto'],
    referencia_clusters['tema_de_referencia']
)
display(tabla_clusters)

for grupo in sorted(resultados_agrupamiento['cluster_propuesto'].unique()):
    print(f'\n(grupo): ejemplos para revisión')
    display(resultados_agrupamiento.query('cluster_propuesto == @grupo')[['origen', 'texto']].head(4))

resultados_agrupamiento.to_csv('resultados_agrupamiento.csv', index=False, encoding='utf-8')
```

tema_de_referencia	cuenta	entregas	pagos
cluster_propuesto			
Grupo 1	2	0	10
Grupo 2	1	15	5
Grupo 3	12	0	0

Grupo 1: ejemplos para revisión

	origen	texto
10	semilla revisada	El cobro de mi pedido apareció dos veces en la...
12	semilla revisada	La tarjeta fue rechazada aunque tiene saldo di...
13	semilla revisada	Pagué por transferencia y el pedido sigue pend...
14	semilla revisada	La factura tiene un valor diferente al que mos...

Grupo 2: ejemplos para revisión

	origen	texto
0	semilla revisada	El seguimiento del pedido no cambia desde hace...
1	semilla revisada	Mi compra aparece enviada, pero el rastreo no ...
2	semilla revisada	El mensajero marcó la entrega como realizada y...
3	semilla revisada	Necesito cambiar la dirección porque el pedido...

Grupo 3: ejemplos para revisión

	origen	texto
20	semilla revisada	No me llega el código OTP para entrar a mi cue...
21	semilla revisada	Mi usuario quedó bloqueado después de varios l...
22	semilla revisada	Olvidé la contraseña y el enlace para recupera...
23	semilla revisada	Quiero actualizar el número de teléfono asocia...

Próximos pasos: [Generar código con tabla_clusters](#) [New interactive sheet](#)

Mapa 3D del lote completo

La proyección usa los embeddings de los 60 tickets: 30 semilla, 15 de validación y 15 operativos. Los colores de entregas, pagos y cuenta muestran el tema de referencia o el tema propuesto. Los puntos rojos tienen atención especial: pueden ser un ticket operativo que requiere revisión humana o un ticket de validación cuyo tema propuesto no coincidió con la etiqueta conocida. No son puntos "malos" por estar rojos; sirven para que el equipo sepa cuáles leer antes de tomar una decisión automática.

```
[111]
✓ 1s

# Los 30 casos semilla conservan siempre el tema de referencia.
tema_semilla = [nombres_clase[indice] for indice in etiquetas]

# En validación se conoce la respuesta correcta. Si el modelo falla, se resalta en rojo.
tema_validacion = [nombres_clase[indice] for indice in etiquetas_validacion]
error_validacion = prediccion_validacion != etiquetas_validacion
tema_visual_validacion = np.where(
    error_validacion, 'Error de validación', tema_validacion
).tolist()

# En los casos operativos, el rojo tiene prioridad sobre el tema propuesto.
tema_visual_operativo = resultados_operativos['tema_propuesto'].where(
    resultados_operativos['estado'].eq('Clasificación propuesta'),
    'Revisión humana'
).tolist()
tema_visual = tema_semilla + tema_visual_validacion + tema_visual_operativo
identificadores = {
    ['S{i + 1:02d}' for i in range(len(corpus))] +
    ['V{i + 1:02d}' for i in range(len(tickets_validacion))] +
    ['O{i + 1:02d}' for i in range(len(tickets_operativos))]
}

# PCA no crea embeddings nuevos: comprime los vectores existentes a tres coordenadas visibles.
proyeccion_completa = PCA(n_components=3, random_state=SEED).fit_transform(embeddings_todos)
mapa_completo = pd.DataFrame({
    'x': proyeccion_completa[:, 0],
    'y': proyeccion_completa[:, 1],
    'z': proyeccion_completa[:, 2],
    'tema_o_estado': tema_visual,
    'origen': origen_todos,
    'ticket': identificadores,
    'texto': textos_todos,
})
print(
    f'Mapa completo: {len(mapa_completo)} puntos | '
    f'errores de validación: {error_validacion.sum()} | '
    f'casos para revisión humana: {requiere_revision.sum()}'
)

# Este gráfico estático es el respaldo: debe verse en Colab y en VS Code aunque Plotly no cargue.
colores_tema = {
    'entregas': '#2E8B57', 'pagos': '#D99022', 'cuenta': '#7655A5',
    'Revisión humana': '#C62828', 'Error de validación': '#C62828'
}
figura_estatica = plt.figure(figsize=(10, 7))
ejes_3d = figura_estatica.add_subplot(111, projection='3d')
for tema, puntos_del_tema in mapa_completo.groupby('tema_o_estado'):
    ejes_3d.scatter(
        puntos_del_tema['x'], puntos_del_tema['y'], puntos_del_tema['z'],
        s=55, color=colores_tema[tema], label=tema, alpha=0.85, edgecolors='white'
    )
ejes_3d.set_title('Mapa 3D completo: 60 tickets y casos por verificar')
ejes_3d.set_xlabel('Componente 1')
ejes_3d.set_ylabel('Componente 2')
ejes_3d.set_zlabel('Componente 3')
ejes_3d.view_init(elev=22, azim=45)
ejes_3d.legend(title='Tema o estado', loc='upper left', bbox_to_anchor=(1.02, 1))
# Se etiqueta solo lo rojo para no tapar los 60 puntos con texto.
puntos_rojos = mapa_completo[mapa_completo['tema_o_estado'].isin(['Revisión humana', 'Error de validación'])]
for _, punto in puntos_rojos.iterrows():
    ejes_3d.text(punto['x'], punto['y'], punto['z'], punto['ticket'], color='#8B0000', fontsize=8)
plt.tight_layout()
plt.show()

# La versión Plotly permite girar el gráfico y leer cada ticket al pasar el cursor.
figura_completa = px.scatter_3d(
    mapa_completo, x='x', y='y', z='z', color='tema_o_estado', symbol='origen',
    hover_name='ticket',
    hover_data={'texto': True, 'origen': True, 'x': ':.3f', 'y': ':.3f', 'z': ':.3f'},
    color_discrete_map=colores_tema,
    title='Mapa 3D completo: 60 tickets y casos por verificar',
)
figura_completa.update_traces(marker={'size': 5})
figura_completa.update_layout(
    scene={'axis_title': 'Componente 1', 'yaxis_title': 'Componente 2', 'zaxis_title': 'Componente 3'}
)
# La visualización interactiva se muestra sola al final del notebook.

# La tabla queda después del gráfico para consultar las coordenadas y el texto completo.
display(mapa_completo)
```

ze	0.00000	-0.20991	-0.409010	cuenta	semilla revisada	S29	Quedaron dos permios creados con el mismo doc...
29	0.313520	-0.060146	-0.259546	cuenta	semilla revisada	S30	El nombre que aparece en mi perfil está escrit...
30	-0.253645	0.384126	-0.109794	entregas	validación	V01	El rastreo sigue detenido y no sé cuándo llega...
31	0.107093	0.184672	0.231066	entregas	validación	V02	La plataforma indicó que mi pedido fue entrega...
32	-0.220915	0.090016	0.291045	entregas	validación	V03	El envío quedó devuelto al vendedor sin que yo...
33	-0.302554	0.065938	-0.352918	entregas	validación	V04	El paquete lleva una semana en la misma ciudad.
34	0.127790	0.436456	0.004772	entregas	validación	V05	Necesito confirmar si todavía puedo modificar ...

35	-0.448052	-0.395790	-0.336358	pagos	validación	V06	Veo un débito duplicado por la misma compra.
36	-0.065586	-0.289236	0.280528	pagos	validación	V07	El reembolso prometido no se refleja en mi cue...
37	-0.345479	-0.258406	-0.091478	pagos	validación	V08	El total final cambió cuando escogí pagar con ...
38	-0.190851	-0.191697	0.355815	pagos	validación	V09	La entidad autorizó el pago, pero la tienda lo...
39	-0.163960	-0.152475	0.303158	pagos	validación	V10	El pedido figura sin pagar pese a que recibí e...
40	0.504326	-0.114378	0.245577	cuenta	validación	V11	No recibo el mensaje con el código para ingresar.
41	0.144034	-0.021118	-0.014727	cuenta	validación	V12	Mi cuenta se bloqueó y no puedo hacer compras.
42	0.351473	0.008439	-0.234648	cuenta	validación	V13	El enlace para cambiar mi clave ya venció.
43	0.566307	0.073653	-0.043962	cuenta	validación	V14	No puedo iniciar sesión desde el celular nuevo.
44	0.200567	-0.005634	-0.115572	cuenta	validación	V15	Aparece un inicio de sesión que yo no realicé.
45	0.167780	0.410874	0.081277	Revisión humana	operativo sin etiqueta	OP01	Mi pedido sale como despachado desde ayer, per...
46	-0.213910	-0.078159	-0.024886	Revisión humana	operativo sin etiqueta	OP02	El repartidor dejó el paquete en otra direcció...
47	-0.288424	0.359008	0.017481	Revisión humana	operativo sin etiqueta	OP03	¿Puedo cambiar el lugar de entrega antes de qu...
48	-0.026464	-0.018524	-0.202707	Revisión humana	operativo sin etiqueta	OP04	Me faltó un artículo dentro de la caja que rec...
49	-0.078841	0.513277	0.033144	Revisión humana	operativo sin etiqueta	OP05	El envío tiene retraso y nadie me confirma una...
50	-0.316047	-0.370893	-0.388687	Revisión humana	operativo sin etiqueta	OP06	Me cobraron dos veces el mismo producto.
51	-0.179771	0.047563	-0.028277	Revisión humana	operativo sin etiqueta	OP07	Devolví el artículo y todavía no aparece el di...
52	-0.243331	-0.243310	0.386956	Revisión humana	operativo sin etiqueta	OP08	El valor cobrado no coincide con el resumen de...
53	-0.338566	-0.076363	0.054369	Revisión humana	operativo sin etiqueta	OP09	Mi transferencia fue exitosa, pero el pago sig...
54	-0.318672	-0.029005	0.230530	Revisión humana	operativo sin etiqueta	OP10	El pedido quedó pendiente después de que reali...
55	0.563074	-0.100178	0.011414	Revisión humana	operativo sin etiqueta	OP11	La clave nueva no me permite entrar a la aplic...
56	0.381203	-0.282637	0.235894	Revisión humana	operativo sin etiqueta	OP12	El código para validar la cuenta nunca llega a...
57	0.484358	-0.032420	-0.362604	Revisión humana	operativo sin etiqueta	OP13	Necesito sacar un dispositivo desconocido de m...
58	0.509148	0.024381	0.185587	Revisión humana	operativo sin etiqueta	OP14	No puedo cambiar el correo asociado a mi usuario.
59	-0.015568	0.177721	0.340487	Revisión humana	operativo sin etiqueta	OP15	El pedido cambió de dirección y ahora el pago ...

Próximos pasos: [Generar código con mapa_completo](#) [New interactive sheet](#)

8. Visualización final: mapa completo de 60 tickets

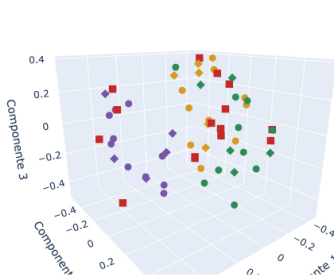
Esta es la gráfica que resume el ejercicio. Cada punto representa una frase de los 60 tickets. Los colores verde, amarillo y morado corresponden a entregas, pagos y cuenta. Los puntos rojos muestran los tickets que requieren atención: una propuesta operativa con poca seguridad o un caso de validación que el clasificador no acertó. Pasa el cursor sobre un punto para leer su identificador, origen y texto.

No aparece una tabla después de esta celda a propósito: el último resultado visible debe ser el gráfico 3D completo.

```
[12] # El objeto ya contiene los 60 puntos creados en la sección anterior.
✓ 0 s print(f'Mostrando {len(mapa_completo)} tickets en el mapa 3D completo.')
# 'show()' funciona de forma directa en el entorno donde se ejecutó el mini ejemplo 01-06.
figura_completa.show()
```

Mostrando 60 tickets en el mapa 3D completo.

Mapa 3D completo: 60 tickets y casos por verificar



tema_o_estado, origen

- entregas, semilla revisada
- entregas, validación
- pagos, semilla revisada
- pagos, validación
- cuenta, semilla revisada
- cuenta, validación
- Revisión humana, operativo sin etiqueta

Conclusión

La tokenización prepara el texto; el embedding aporta una representación densa para que un modelo pueda aprender relaciones. En este ejemplo, los embeddings no reemplazan al equipo de servicio: ayudan a proponer un tema, agrupar solicitudes parecidas y mostrar cuáles casos sería mejor revisar antes de usarlos para una ruta automática.